Smart Agriculture and Greenhouse IoT Controller Systems

1 INTRODUCTION

The **Smart Agriculture & Greenhouse IoT Controller System** is a software simulation and monitoring platform for managing an automated greenhouse environment. The system collects environmental data from sensors such as soil-moisture, temperature, and sunlight sensors and uses this information to monitor growing conditions and control greenhouse equipment. Based on configurable rules and control strategies, the system can activate actuators such as irrigation pumps, heating systems, and automated shade curtains. The system models sensors, actuators, greenhouse zones, environmental rules, and control strategies as interacting objects. The project provides an opportunity to apply object-oriented principles and design patterns to an event-driven automation system.

2 OBJECTIVE OVERVIEW

The primary objective is to design and develop an object-oriented greenhouse automation system capable of monitoring environmental conditions and automatically responding to changing conditions. Students will model physical devices as software objects and establish relationships between sensors, actuators, greenhouse zones, rules, and control strategies. The project should demonstrate **abstraction, inheritance, polymorphism, encapsulation, and composition**. Students will also apply the **Observer Pattern** for distributing sensor updates, the **Strategy Pattern** for interchangeable greenhouse control policies, and the **Composite Pattern** for organizing devices into modular zones. The final design should demonstrate how object-oriented techniques can be used to create an extensible and maintainable event-driven automation system.

3 DETAILED FUNCTIONAL REQUIREMENTS

3.1 Greenhouse and Zone Management

The system shall provide a virtual greenhouse containing one or more independently managed zones.

Users shall be able to:

- Create and configure a greenhouse.
- Add or remove greenhouse zones.
- View the current status of each zone.
- Configure environmental targets for individual zones.
- Add sensors and actuators to appropriate zones.
- Monitor all zones or inspect an individual zone.

A **Zone** shall act as a composite object containing multiple sensors and actuators. The Composite Pattern should allow individual devices and groups of devices to be managed through a consistent interface where appropriate.

3.2 Sensor Management

The system shall support multiple types of environmental sensors.

At minimum, the system should model:

- Soil moisture sensors.
- Ambient temperature sensors.
- Sunlight/light sensors.

Additional sensor types may be introduced where appropriate.

All sensors should share common characteristics through a generic Sensor abstraction while specialized sensors provide their own measurement behavior.

Each sensor should maintain information such as:

- Sensor identifier.
- Sensor type.
- Current measurement.
- Measurement unit.
- Operational status.
- Associated greenhouse zone.
- Measurement/update time.

Sensor readings may change as the simulation progresses.

3.3 Sensor Data Monitoring

The system shall continuously or periodically obtain sensor measurements.

Users should be able to:

- View current sensor readings.
- View the status of sensors.
- Monitor historical measurements where persistence is implemented.
- Identify measurements outside configured acceptable ranges.
- Observe changes in environmental conditions.

The system should notify interested components whenever relevant sensor values change.

The **Observer Pattern** shall be used to establish a broadcaster/listener relationship between sensor data sources and interested components such as user-interface views, monitoring services, and rule evaluators.

A sensor should not need direct knowledge of all components that consume its data.

3.4 Actuator Management

The system shall support different types of greenhouse actuators.

At minimum, the system should model:

- Irrigation pumps.
- Heating coils or heating systems.
- Automated shade curtains.

Additional actuators may be supported.

All actuators should share a common abstraction while specialized actuator classes implement device-specific behavior.

The system shall allow authorized users or automation rules to:

- Activate an actuator.
- Deactivate an actuator.
- Inspect actuator status.
- Configure actuator parameters where appropriate.
- Associate actuators with greenhouse zones.

3.5 Rule-Based Environmental Control

The system shall provide a mechanism for evaluating environmental conditions and determining whether an automated action is required.

Examples include:

- Activate irrigation when soil moisture falls below a threshold.
- Activate heating when temperature falls below a minimum value.
- Close shade curtains when sunlight exceeds a specified level.
- Deactivate an actuator when the environmental condition returns to an acceptable range.

Rules should contain appropriate conditions and actions and should be evaluated when relevant sensor events occur.

The rule engine should interact with sensors and actuators through abstractions rather than depending on specific device implementations.

3.6 Polymorphic Rule Processing

Different environmental rules may require different evaluation logic.

The system should provide a common rule abstraction with specialized implementations for conditions such as:

- Low soil moisture.
- High temperature.
- Low temperature.
- Excessive sunlight.
- Combined environmental conditions.

The rule engine should process rules polymorphically so that new rule types can be added without modifying the core rule-processing mechanism.

Students should avoid implementing the rule engine as a large collection of conditional statements based on sensor type.

3.7 Control Strategies

The system shall support multiple greenhouse control strategies.

At minimum, the following strategies should be considered:

- **EcoMode** — attempts to minimize resource consumption.
- **HighYieldGrowth** — prioritizes environmental conditions favorable to crop growth.
- **FrostProtection** — prioritizes protection against low-temperature conditions.

The **Strategy Pattern** shall be used so that the active control strategy can be changed without modifying the greenhouse or rule-engine classes.

Different strategies may make different decisions when presented with the same environmental conditions.

For example, EcoMode may delay irrigation to conserve water, while HighYieldGrowth may irrigate earlier to maintain optimal soil moisture.

3.8 Event-Driven Automation

The system shall operate in an event-driven manner.

When a sensor reading changes:

1. The sensor generates an update event.
2. Registered observers receive the notification.
3. Monitoring or rule components evaluate the new environmental condition.
4. The active control strategy determines the appropriate response.
5. Relevant actuators are activated, deactivated, or reconfigured.
6. The user interface is updated to reflect the new system state.

The design should minimize direct dependencies between these components.

3.9 Manual Control

The system should provide users with the ability to manually control actuators when appropriate.

Users should be able to:

- Turn an actuator on or off.
- Override an automated action.
- Return the actuator to automatic control.
- View whether an actuator is currently under manual or automatic control.

The system should clearly distinguish between manual commands and automated commands.

3.10 Alerts and Notifications

The system should provide notifications when significant environmental or system conditions occur.

Examples include:

- Soil moisture critically low.
- Temperature below a safe threshold.
- Temperature above a safe threshold.
- Excessive sunlight.
- Sensor failure.
- Actuator failure.
- Conflicting control conditions.

Notifications should be delivered to appropriate observers without requiring the sensor or rule engine to directly manage user-interface components.

3.11 Simulation and Time Progression

Because this is a simulation system, the application shall provide a mechanism for advancing or continuously running virtual time.

As time progresses:

- Sensor values may change.
- Environmental conditions may evolve.
- Rules may be triggered.
- Actuators may operate for specified durations.
- Greenhouse zones may transition between different states.

The simulation should provide sufficient control to demonstrate automated behavior during project evaluation.

3.12 System Monitoring and Dashboard

The system shall provide an overview of greenhouse operation.

The dashboard should display, where applicable:

- Current greenhouse zones.
- Sensor readings.
- Sensor status.
- Active actuators.
- Current control strategy.
- Triggered rules.
- Alerts and notifications.
- Recent environmental changes.
- Manual overrides.

The interface should allow users to understand why an automated action was performed.

4 TECHNICAL ARCHITECTURE AND DATABASE REQUIREMENTS

The system should follow a modular architecture separating the user interface, event-processing layer, domain model, automation/rule engine, and persistence layer.

A possible architecture is:

User Interface → Monitoring/Event Layer → Rule Engine → Control Strategy → Actuator Layer

The domain model should contain abstractions such as Sensor, Actuator, Rule, ControlStrategy, and GreenhouseComponent, together with specialized implementations.

The **Observer Pattern** should connect sensors with monitoring and rule-processing components. The **Strategy Pattern** should encapsulate alternative greenhouse control policies. The **Composite Pattern** should organize sensors and actuators into Zone objects and support consistent management of individual components and groups.

If persistent storage is implemented, the database should store greenhouse configurations, zones, sensors, actuators, sensor readings, rules, strategies, alerts, and actuator events. Appropriate primary keys, foreign keys, timestamps, and relationships should be defined. Historical sensor readings should be separated from current device state where appropriate.

The architecture should support adding new sensor types, actuator types, rules, and control strategies without requiring major changes to existing components.

5 CHALLENGES AND CONSTRAINTS

The primary challenge is coordinating multiple objects in an event-driven environment while maintaining low coupling between components. Students must ensure that sensor updates can trigger appropriate actions without creating direct dependencies between every sensor, rule, and actuator. Designing meaningful abstractions for heterogeneous devices is another challenge. The system must also handle conflicting rules, manual overrides, sensor failures, and changing environmental conditions. Students should apply the Observer, Strategy, and Composite Patterns to genuine architectural problems rather than simply including them for demonstration. The project should remain sufficiently simple to implement within the course schedule while still demonstrating meaningful object-oriented design.

6 CONCLUSION

The Smart Agriculture & Greenhouse IoT Controller System provides a practical domain for applying object-oriented analysis and design to an event-driven automation problem. Students will model sensors, actuators, rules, zones, and control strategies as collaborating objects. The project emphasizes abstraction, polymorphism, composition, and three complementary design patterns while demonstrating how software objects can represent and automate a physical environment.